

Strings

Strings are defined as an array of characters. The difference between a character array and a string is the string is terminated with a special character '\0' or Strings are considered a **data type** in general and are typically represented as arrays of bytes (or words) that store a sequence of characters.

```
#include <stdio.h>

int main()
{
    // declare and initialize string
    char str[] = "hello";

    // print string
    printf("%s", str);

    return 0;
}
```

must-know concepts of string:

- Accessing characters by index in a string.
- Inserting character into a String.
- Modifying character in String
- Deletion of Character in String
- Concatenating strings (combining multiple strings into one).
- Finding the length of a string
- Comparing strings for equality or lexicographical order

Methods on strings operation

Concatenation of Strings

The process of combining more than one string together is known as Concatenation. String Concatenation is the technique of combining two strings.

CONCATENATE (STR1, STR2, STR3)

1. LEN1 = LENGTH(STR1).
2. LEN2 = LENGTH(STR2).
3. SET I = 0.
4. Repeat Steps 5 and 6 while I < LEN1-1:
5. STR3[I] = STR1[I].
6. SET I = I+1.

7. SET J = 0.
8. Repeat Steps 9 to 11 while I < (LEN1 + LEN2 - 2):
9. STR3[I] = STR2[J].
10. J = J+1.
11. I = I+1.
- 12.Exit.

```
#include <stdio.h>
int main() {
    char s1[100] = "programming ", s2[] = "is awesome";
    int length, j;

    // store length of s1 in the length variable
    length = 0;
    while (s1[length] != '\0') {
        ++length;
    }

    // concatenate s2 to s1
    for (j = 0; s2[j] != '\0'; ++j, ++length) {
        s1[length] = s2[j];
    }

    // terminating the s1 string
    s1[length] = '\0';

    printf("After concatenation: ");
    puts(s1);

    return 0;
}
```

C program for comparison for 2 strings (char array) strcmp

```
#include <stdio.h>
#include <string.h>
int main()
{
    char str1[50]; // declaration of char array
    char str2[50]; // declaration of char array
    int value; // declaration of integer variable

    printf("Enter the first string : ");
    scanf("%s", str1);
```

```

printf("Enter the second string : ");
scanf("%s",str2);

// comparing both the strings using strcmp() function
value = strcmp(str1,str2);
if(value == 0)
printf("strings are same");
else
printf("strings are not same");
return 0;
}

```

Function: Strchr

```

const char * strchr ( const char * str, int character );    char * strchr ( char * str,
int character );

```

Locate first occurrence of character in string

Returns a pointer to the first occurrence of *character* in the C string *str*.

this function is only declared as:

```

char * strchr ( const char *, int );

```

```

/* strchr example */
#include <stdio.h>
#include <string.h>
int main ()
{
    char str[] = "This is a sample string";
    char * pch;
    printf ("Looking for the 's' character in \"%s\"...\n",str);
    pch=strchr(str,'s');
    while (pch!=NULL)
    {
        printf ("found at %d\n",pch-str+1);
        pch=strchr(pch+1,'s');
    }
    return 0;
}

```

Function: strpbrk

```
const char * strpbrk ( const char * str1, const char * str2 );    char * strpbrk (
char * str1, const char * str2 );
```

Locate characters in string

Returns a pointer to the first occurrence in *str1* of any of the characters that are part of *str2*, or a null pointer if there are no matches.

The search does not include the terminating null-characters of either strings, but ends there.

In C, this function is only declared as:

```
char * strpbrk ( const char *, const char * );
```

```
/* strpbrk example */
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main ()
```

```
{
```

```
    char str[] = "This is a sample string";
```

```
    char key[] = "aeiou";
```

```
    char * pch;
```

```
    printf ("Vowels in '%s': ",str);
```

```
    pch = strpbrk (str, key);
```

```
    while (pch != NULL)
```

```
    {
```

```
        printf ("%c ", *pch);
```

```
        pch = strpbrk (pch+1,key);
```

```
    }
```

```
    printf ("\n");
```

```
    return 0;
```

```
}
```

Output: Vowels in 'This is a sample string': i i a a e i

Function: strstr

```
const char * strstr ( const char * str1, const char * str2 );    char * strstr (
char * str1, const char * str2 );
```

Locate substring

Returns a pointer to the first occurrence of *str2* in *str1*, or a null pointer if *str2* is not part of *str1*.

The matching process does not include the terminating null-characters, but it stops there.

In C, this function is only declared as:

```
char * strstr ( const char *, const char * );
```

```
/* strstr example */  
#include <stdio.h>  
#include <string.h>
```

```
int main ()  
{  
    char str[] = "This is a simple string";  
    char * pch;  
    pch = strstr (str, "simple");  
    if (pch != NULL)  
        strncpy (pch, "sample", 6);  
    puts (str);  
    return 0;  
}
```

This example searches for the "simple" substring in *str* and replaces that word for "sample".

Function: strspn

```
size_t strspn ( const char * str1, const char * str2 );
```

Get span of character set in string

Returns the length of the initial portion of *str1* which consists only of characters that are part of *str2*.

The search does not include the terminating null-characters of either strings, but ends there.

The length of the initial portion of *str1* containing only characters that appear in *str2*.

Therefore, if all of the characters in *str1* are in *str2*, the function returns the length of the entire *str1* string, and if the first character in *str1* is not in *str2*, the function returns zero.

```
/* strspn example */
```

```

#include <stdio.h>
#include <string.h>

int main ()
{
    int i;
    char strtex[] = "129th";
    char cset[] = "1234567890";

    i = strspn (strtex,cset);
    printf ("The initial number has %d digits.\n",i);
    return 0;
}

```

Function : strcspn

```
size_t strcspn ( const char * str1, const char * str2 );
```

Get span until character in string

Scans *str1* for the first occurrence of any of the characters that are part of *str2*, returning the number of characters of *str1* read before this first occurrence.

The search includes the terminating null-characters. Therefore, the function will return the length of *str1* if none of the characters of *str2* are found in *str1*.

```

* strcspn example */
#include <stdio.h>
#include <string.h>

int main ()
{
    char str[] = "fcba73";
    char keys[] = "1234567890";
    int i;
    i = strcspn (str,keys);
    printf ("The first number in str is at position %d.\n",i+1);
    return 0;
}

```

The first number in str is at position 5

Concept of ADT in Data structures

an **abstract data type (ADT)** is a [mathematical model](#) for [data types](#), defined by its behavior ([semantics](#)) from the point of view of a [user](#) of the data, specifically in terms of possible values, possible operations on data of this type, and the behavior of these operations. This mathematical model contrasts with [data structures](#), which are concrete representations of data, and are the point of view of an implementer, not a user. For example, a [stack](#) has push/pop operations that follow a Last-In-First-Out rule, and can be concretely implemented using either a list or an array. Another example is a [set](#) which stores values, without any particular [order](#), and no repeated values. Values themselves are not retrieved from sets, rather one tests a value for membership to obtain a Boolean "in" or "not in"

In C, you can create a simple abstract data type (ADT) for natural numbers using a structure and functions to operate on that structure. Here's an example:

ADT for complex numbers in c

```
#include <stdio.h>
```

```
#include <math.h>
```

```
// Define the structure for complex numbers
```

```
typedef struct {
```

```
    double real;
```

```
    double imag;
```

```
} Complex;
```

```
// Function to create a complex number
```

```
Complex createComplex(double real, double imag) {
```

```
    Complex result;
```

```
    result.real = real;
```

```
    result.imag = imag;
```

```
    return result;
```

```
}
```

// Function to add two complex numbers

Complex addComplex(Complex a, Complex b) {

Complex result;

result.real = a.real + b.real;

result.imag = a.imag + b.imag;

return result;

}

// Function to subtract two complex numbers

Complex subtractComplex(Complex a, Complex b) {

Complex result;

result.real = a.real - b.real;

result.imag = a.imag - b.imag;

return result;

}

// Function to multiply two complex numbers

Complex multiplyComplex(Complex a, Complex b) {

Complex result;

result.real = a.real * b.real - a.imag * b.imag;

result.imag = a.real * b.imag + a.imag * b.real;

return result;

}

// Function to calculate the magnitude of a complex number

double magnitudeComplex(Complex num) {

return sqrt(num.real * num.real + num.imag * num.imag);


```
}
```

```
// Function to print a complex number
```

```
void printComplex(Complex num) {  
    printf("%.2f + %.2fi\n", num.real, num.imag);  
}
```

```
int main() {
```

```
    // Example usage
```

```
    Complex num1 = createComplex(2.0, 3.0);
```

```
    Complex num2 = createComplex(1.0, -1.0);
```

```
    printf("Complex Number 1: ");
```

```
    printComplex(num1);
```

```
    printf("Complex Number 2: ");
```

```
    printComplex(num2);
```

```
    Complex sum = addComplex(num1, num2);
```

```
    printf("Sum: ");
```

```
    printComplex(sum);
```

```
    Complex difference = subtractComplex(num1, num2);
```

```
    printf("Difference: ");
```

```
    printComplex(difference);
```

```
    Complex product = multiplyComplex(num1, num2);
```

```

printf("Product: ");
printComplex(product);

printf("Magnitude of Complex Number 1: %.2f\n",
magnitudeComplex(num1));

return 0;
}

```

ADT for natural numbers

Refer ppt forwarded.

Difference between structures and unions

	STRUCTURE	UNION
Keyword	The keyword struct is used to define a structure	The keyword union is used to define a union.
Size	When a variable is associated with a structure, the compiler allocates the memory for each member. The size of structure is greater than or equal to the sum of sizes of its members.	when a variable is associated with a union, the compiler allocates the memory by considering the size of the largest memory. So, size of union is equal to the size of largest member.
Memory	Each member within a structure is assigned unique storage area of location.	Memory allocated is shared by individual members of union.
Value Altering	Altering the value of a member will not affect other members of the structure.	Altering the value of any of the member will alter other member values.
Accessing members	Individual member can be accessed at a time.	Only one member can be accessed at a time.
Initialization of Members	Several members of a structure can initialize at once.	Only the first member of a union can be initialized.

Similarities Between Structure and Union

1. Both are user-defined data types used to store data of different types as a single unit.
2. Their members can be objects of any type, including other structures and unions or [arrays](#). A member can also consist of a bit field.
3. Both structures and unions support only assignment = and [sizeof](#) operators. The two structures or unions in the assignment must have the same members and member types.
4. A structure or a union can be passed by value to [functions](#) and returned by value by functions. The argument must have the same type as the function

parameter. A structure or union is passed by value just like a scalar variable as a corresponding parameter.

5. ‘.’ [operator](#) or selection operator, which has one of the highest precedences, is used for accessing member variables inside both the user-defined datatypes

Refer text book for example program. Or any online site which is genuine.